The Journal of Open Source Software

SynthEddy: A Python program for generating synthetic turbulent flow fields

Ange (Phil) Du [1], **Nikita Holyev**[1], **and Spencer Smith** [1]

1 McMaster University, Canada

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 02 July 2026
Published: unpublished

Summary

Turbulent flow is characterized by chaotic fluctuations in velocity and pressure. Today, most turbulent computational fluid dynamic (CFD) simulations are based on simplified models, such as the Reynolds-Averaged Navier-Stokes (RANS) equations. While other approaches can reveal more details in the flow, such as Large Eddy Simulation (LES) and Direct Numerical Simulation (DNS), they are often deemed too computationally expensive and thus see limited use in practice. Part of the computational expense is the amount of simulation time needed for turbulence to properly develop from simple initial conditions (IC) and boundary conditions (BC). `SynthEddy` is a Python program that aims to alleviate the start-up computational cost by generating IC and BC for LES using the synthetic eddy method (SEM). The synthetic eddies are constructed to have the required turbulence characteristics, so simulation time is not needed for proper turbulence to emerge. `SynthEddy` reduces the required simulation scale both in time and space, making LES more accessible to a wider audience.

Statement of need

Compared to widely used RANS models in CFD, LES can capture more intricate details of a turbulent flow, down to individual eddies. This can be desirable in many applications or scenarios, such as gas turbine design, airfoil flow separation, construction airflow, ocean studies, and more (Chamecki et al., 2019; Van Maele & Merci, 2008; WON-WOOK KIM & MONGIA, 1999; You et al., 2008). However, the adoption of LES has been largely limited by its computational cost, which can be orders of magnitude higher than RANS (Yang, 2015). Not only the model itself is more computationally expensive, but there exists a compounding issue with the initiation of the simulation.

Unlike RANS, which can be described by a few parameters, utilizing LES in any practical manner requires a realistic turbulent flow field running in the simulation. To reach such a state, either prior simulations with longer time and larger field for the turbulent flow to develop are needed (more expensive), or a suitable inlet condition must be synthesized using turbulence generation methods (Wu, 2017). Poletto et al. (2013) proposed one such method by generating synthetic eddies to mimic a turbulent flow. Compared to previous proposals, such as random fluctuations, this method is divergence free and closer to actual turbulent flow.

`SynthEddy` is a Python implementation based on Poletto's method. SynthEddy models turbulent flow fields consisting of synthetic eddies of various sizes, orientations and intensities. This physical system is shown in Figure 1. The user can generate a turbulent velocity field and query it for use in both turbulence research and as an IC/BC for CFD simulations.

The Journal of Open Source Software

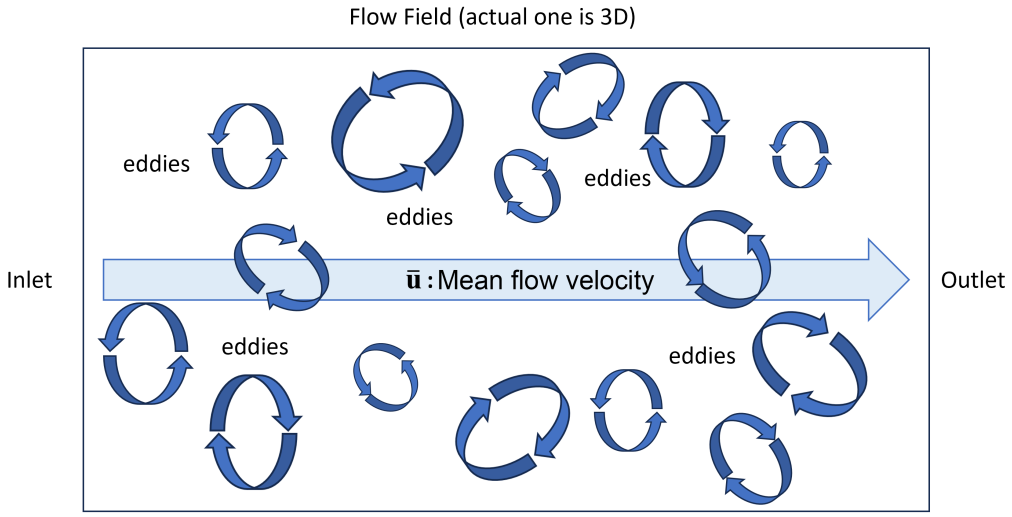

Figure 1: Physical system

State of the field

To obtain the appropriate IC/BC, a CFD practitioner can run an actual experiment or use DNS results, but it may not be feasible for these approaches to match the desired situation. For instance, this would be the case if one desired a DNS at a Reynolds number of 500, but only datasets for Re = 200 are available online. While these datasets could be used, they would need to be rescaled and still would not accurately represent the desired flow conditions. Similarly, if one perform an experiment to study turbulence over turbine blades at a specific Reynolds number, it is difficult to generalize those results to different Reynolds numbers or flow conditions.

This is why we resort to filtering methods, Fourier methods, or coherent methods (the coherent method is the one used in SynthEddy). These approaches are advantageous because they can be adapted to virtually any flow condition while remaining much more computationally efficient than DNS. However, they can lack realistic turbulence physics. For example, the Fourier method generates a velocity field using sine and cosine waves. These waves do not interact with or depend on one another in the same way as structures in real turbulence. The lack of accuracy in the initial conditions will likely hurt the accuracy of the simulation. If your initial conditions from a turbulence generation method lacks realism, it can take longer for the DNS that is being fed these conditions to converge to realistic values, or even to converge at all.

The Synthetic Eddy Method (Jarrin et al., 2006) is an attempt to be a more realistic representation of turbulence, but in SEM the generated eddies do not satisfy continuity (i.e. conservation of mass). The synthetic turbulence can be improved by satisfying continuity. This motivated Poletto et al. (2013) to propose a new divergence-free synthetic eddy method (DFSEM). This is what our paper is based on. Haywood et al. (2021) is a more recent paper that uses a different type of eddies (named Hill's spherical vortex). These eddies look like donuts and the authors claim that this type of structure satisfies Euler's equations. However, the simpler eddies implemented in SynthEddy seem more representative of what is observed in DNS.

Coherent methods (such as SEM or DFSEM) allow one to modify eddies in ways that better represent real turbulent physics. They can resolve the full energy spectrum, prescribe eddy orientations in specific directions, prescribe eddy shape and stretch them, and/or use different eddy densities to better reproduce the turbulent statistics of a wide range of flows (e.g.,

inhomogeneous and anisotropic flows). As a starting point, SynthEddy currently focuses only on reproducing homogeneous isotropic turbulence.

Software design

To ensure `SynthEddy` can adapt to different use cases and be easily maintainable and customizable, the program is designed with the following principles:

Information preserving

A key design philosophy of `SynthEddy` is to preserve as much information as possible in the generated field, hence the separate query process.

Unlike an actual CFD simulation, `SynthEddy` does not use numerical methods to solve the field. Thus, there is no need to discretize the field at the beginning, which can lead to a loss of information. Instead, the eddies are treated as movable individual entities in continuous space and time. The field (or queried region) is only discretized into a meshgrid when queried.

This allows the user to query the same field multiple times with different parameters, such as varying resolution for specific region of interest, or when performing grid sensitivity analysis. When queried at any point in time, the program first finds the location of each eddy at such time analytically, without having to advance through prior time steps numerically.

This opens up the possibility for "salami slicing" the field to obtain BC (as mentioned above) at any arbitrary feeding rate demanded by the user, without concerning grid resolution or time step size.

SynthEddy has been verified following the Verification and Validation Plan, results of which are show in the Verification and Validation Report.

Document driven development

The design and development of `SynthEddy` is driven by various documents in the repository. These documents guide the development process and served as communication bridge between the developer and domain experts. They are also the entry point for future parties looking to perform any significant modification or extension to the program.

Some of the key documents include:

- Software Requirements Specification (SRS)
  - Describes the goals, assumptions and requirements of the program.
  - Documents the mathematical models used to build the program, including their relationships and refinements.
    - One such example for a theoretical model is shown in Figure 2.
- Module Guide (MG)
  - Describes the module architecture of the program.
  - The design of the software is based on the principle of information hiding (David L. Parnas, 1972). That is, the design is decomposed around likely changes, which become the secrets of the modules.
  - Record key design decisions and rationale.
  - The module guide, which includes the uses relation between modules, is based on Parnas's ideas for documenting the software architecture (D. L. Parnas et al., 1984; David L. Parnas & Weiss, October 2001).
- Module Interface Specification
  - The interface provided by each module

  - The MIS follows the approach presented by Hoffman and Strooper (Hoffman & Strooper, 1995) and later adapted for scientific computing software (ElSheikh et al., 2004; Smith & Yu, 2009).
- Verification and Validation Plan (VnV Plan)
  - Describes the testing strategy and test cases.

Number	TM2
Label	**Velocity Fluctuation Field**
Equation	$\mathbf{u}'(\mathbf{x}) = \frac{1}{\sqrt{N}} \sum_{k=1}^{N} q_\sigma(d^k(\mathbf{x}))\mathbf{r}^k(\mathbf{x}) \times \boldsymbol{\alpha}^k$
Description	The equation gives the fluctuation of velocity (the deviation from the mean flow velocity) at point $\mathbf{x}$ in the flow field. It is the normalized sum of the velocity fluctuation by each eddy, with shape function q_σ [TM1]. • N is the number of eddies. • $\mathbf{r}^k$ is the normalized distance to the center of the kth eddy [DD1]. • $\boldsymbol{\alpha}^k$ is the intensity vector of the kth eddy (m/s) [DD4].
Notes	According to TM1, at any single position, the fluctuation from most eddies will likely be 0, due to outside the boundaries of most eddies in the field.
Source	Poletto et al. (2013)
Uses	TM1, DD1, DD4
Ref. By	GD1

Figure 2: Theoretical model example

Continuous integration

SynthEddy uses continuous integration by GitHub actions to call `pytest` for unit and system testing for every pull request on the main branch. This ensures that the program is always in a working state throughout the development process.

These tests can also be run locally with instructions in the README.md. See VnV Plan for more details on the test cases.

Research impact statement

SynthEddy is being used by Nikita Holyev as he works toward his PhD (Holyev et al., 2021). As far as we know, SynthEddy is the only tool that has focused on only the synthetic turbulence. There are other open source tools, like Code_Saturne and OpenFOAM, that have built-in implementations of DFSEM to generate realistic turbulent fluctuations at the inlet of an LES simulation. OpenFOAM has options for digital filter method, divergence-free synthetic turbulence, and random Fourier modes.

Features and usage

- Generate synthetic turbulent flow field consisting of eddies. Users need to provide an eddy profile with the following parameters of each type of eddy to be included in the field:
  - Size (length-scale)
  - Density (number of eddies per unit volume)
  - Intensity magnitude

- Query velocity vectors in the generated field as a meshgrid, which can be:
  - The whole field.
  - Any subsection of the field.
  - At any point in time after generation.

A quick start guide is provided in the README.md of the repository.

The generated field is fully wrapped around on all boundaries, ensuring conservation of mass. Details on how wrapping is handled in different flow scenarios can be found in the Module Guide (MG) (see Field Wrap-around section).

The query result is saved as a NumPy array (`.npy` file) representing velocity vectors in a 3D meshgrid, with a shape of `(Nx, Ny, Nz, 3)` where `N` is the number of grid points in each direction, and the last dimension represents x, y, and z components of the velocity vector.

A velocity magnitude cross-section plot example from a 1000^3 meshgrid is shown in Figure 3.

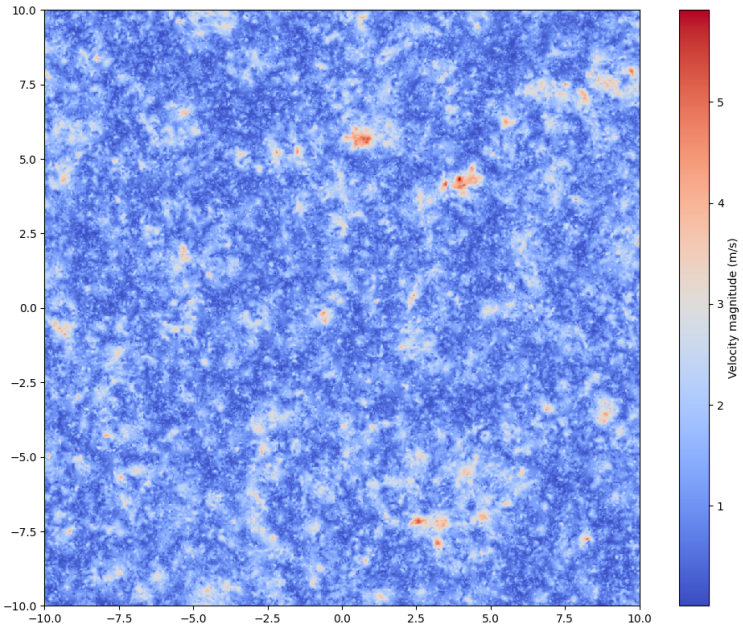

Figure 3: Result cross-section example

Customization

SynthEddy allows the user to easily program certain aspects of the generated field to better suit their specific needs. This includes:

- Non-uniform mean velocity distribution.
  Instead of inputting a constant mean velocity across the field, the user can provide a function to obtain mean x-velocity ($\overline{\mathbf{u}}$) based on y- and z-coordinates. This is useful for simulation of channel flows and boundary layers.
- Eddy shape function. A function that describes the velocity distribution of individual eddies.

  Detailed explanation on editing these functions can be found in the README.md.

Typical use case

- Whole field (IC and research)
  To initiate a CFD simulation, the user can generate a field and query its entirety at $t = 0$ to obtain the initial condition. This use case can also be used in turbulent research, such as turbulent energy spectrum analysis. Depending on the size of the field and query meshgrid resolution, this can take significant time and memory.
- Salami slicing (BC)
  In a continuously running CFD simulation, the user can keep querying a thin subsection of the field with advancing time, and feed the results as inlet boundary conditions to the simulation. Since the region is much smaller than the whole field, this can be done significantly faster.
  `SynthEddy` allows querying any subsection of the same field at any time, and ensures continuity between different queried space and time. See Information preserving for more details.

Performance and benchmark

To improve performance on large meshgrid, the grid is divided into chunks for efficient batch processing. This is detailed in the Module Guide (MG) (see Chunking section). A benchmark test is included in the repository (see README.md). On an Intel i9-13900K CPU, with a 1000^3 meshgrid and around 10 million eddies, the run time is approximately 1 hour.

AI Usage Disclosure

AI was not used for coding or documentation. However, AI was used for brainstorming content for the CONTRIBUTING.md guide, the CodeOfConduct.md and the continuous deployment of the documentation via a GitHub Action. In these cases the tool used was ChatGPT based on GPT-5.5. In addition, Claude (Sonnet 5) was used to modify the human-written Makefile so that it is operating system agnostic. The human authors have reviewed, verified and edited all AI suggested text and scripts.

Acknowledgements

We acknowledge the insights and suggestions from Dr. Marilyn Lightstone and Dr. Stephen Tullis during the development of this program and the preparation of this paper.

References

Chamecki, M., Chor, T., Yang, D., & Meneveau, C. (2019). Material transport in the ocean mixed layer: Recent developments enabled by large eddy simulations. *Reviews of Geophysics*, *57*(4), 1338–1371. https://doi.org/https://doi.org/10.1029/2019RG000655

ElSheikh, A. H., Smith, W. S., & Chidiac, S. E. (2004). Semi-formal design of reliable mesh generation systems. *Advances in Engineering Software*, *35*(12), 827–841.

Haywood, J. S., Sescu, A., Bhushan, S., & Kees, C. (2021). Triple hill's vortex synthetic eddy method. *Flow, Turbulence and Combustion*, *108*, 627–659.

Hoffman, D. M., & Strooper, P. A. (1995). *Software design, automated testing, and maintenance: A practical approach*. International Thomson Computer Press.

Holyev, N., Lightstone, M., & Tullis, S. (2021). Construction of an energy spectrum in a synthetic turbulence generator. *29th Annual Conference of the Computational Fluid Dynamics Society of Canada (CFDSC)*.

Jarrin, N., Benhamadouche, S., Laurence, D., & Prosser, R. (2006). A synthetic-eddy-method for generating inflow conditions for large-eddy simulations. *International Journal of Heat Fluid Flow*, *27*, 585–593.

Parnas, David L. (1972). On the criteria to be used in decomposing systems into modules. *Comm. ACM*, *15*(2), 1053–1058.

Parnas, D. L., Clement, P. C., & Weiss, D. M. (1984). The modular structure of complex systems. *International Conference on Software Engineering*, 408–419.

Parnas, David L., & Weiss, D. M. (October 2001). Definition and documentation of software architectures. *Technical Report ALR-2001-023*.

Poletto, R., Craft, T., & Revell, A. (2013). A new divergence-free synthetic–eddy method for the reproduction of inlet flow conditions for LES. *Flow, Turbulence and Combustion*, *91*(519–539). https://doi.org/10.1007/s10494-013-9488-2

Smith, W. S., & Yu, W. (2009). A document driven methodology for improving the quality of a parallel mesh generation toolbox. *Advances in Engineering Software*, *40*(11), 1155–1167. https://doi.org/http://dx.doi.org/10.1016/j.advengsoft.2009.05.003

Van Maele, K., & Merci, B. (2008). Application of RANS and LES field simulations to predict the critical ventilation velocity in longitudinally ventilated horizontal tunnels. *Fire Safety Journal*, *43*(8), 598–609. https://doi.org/https://doi.org/10.1016/j.firesaf.2008.02.002

WON-WOOK KIM, S. M., & MONGIA, H. C. (1999). Large-eddy simulation of a gas turbine combustor flow. *Combustion Science and Technology*, *143*(1-6), 25–62. https://doi.org/10.1080/00102209908924192

Wu, X. (2017). Inflow turbulence generation methods. *Annual Reviews of Fluid Mechanics*, *49*, 23–49.

Yang, Z. (2015). Large-eddy simulation: Past, present and the future. *Chinese Journal of Aeronautics*, *28*(1), 11–24. https://doi.org/https://doi.org/10.1016/j.cja.2014.12.007

You, D., Ham, F., & Moin, P. (2008). Discrete conservation principles in large-eddy simulation with application to separation control over an airfoil. *Physics of Fluids*, *20*(10), 101515. https://doi.org/10.1063/1.3006077